

OSB-Rastreator

What is OSB-Rastreator?

- OSB-Rastreator is a tool that allows for the automatic analysis of open-source software code, searching for potential vulnerabilities.

```
@mail:~/perro$ cat get_list_openSource.txt
apt list | cut -d'/' -f1 >> /home/pablo/perro/open2.txt
@mail:~/perro$ wc -l openHack.txt
45494 openHack.txt
@mail:~/perro$
```

Source: elladodelmal.com from <https://www.elladodelmal.com/2015/03/osb-rastreator-como-localizar.html>

```
~ » apt list 2>/dev/null | wc -l
84960
```

Picture own elaboration

OSB Rastreator es una herramienta que permite analizar automáticamente el código fuente de software libre en busca de posibles vulnerabilidades.

Esta herramienta fue desarrollada por Pablo González en 2015 como respuesta a la gran cantidad de software vulnerable presente en los repositorios de código abierto de Linux.

Por aquel entonces Pablo se dio cuenta de que existían gran cantidad de repositorios a nuestro alcance a partir del simple comando `apt-get install` y sin embargo se preguntaba si todos los paquetes, especialmente el código, estaba incorrectamente auditado y revisado en busca de vulnerabilidades.

Para resolver su duda lo que hizo fue ejecutar los comandos que podéis ver en la imagen y luego posteriormente con `wc -l` llegó a la conclusión de que en Ubuntu en aquel momento existían más de 45 mil paquetes que a golpe de comando podríamos instalar en nuestro dispositivo.

En estos comandos lo que hacía es utilizar `apt list`, que lo que nos muestra son todos los paquetes disponibles, en este caso en Ubuntu, y luego con `cat -d` lo que hacía era hacer un split en ese string que obtenía como respuesta del comando y quedarse con la primera parte de ese split, que lo que era es el nombre del paquete.

Posteriormente ese nombre lo guardaba en el fichero, en este caso openHack.txt, el cual luego con `wc -l` (word counter) leía cuántas líneas tenía y por tanto cada línea era un paquete, pues había 45.494 paquetes.

Hoy en día en una distribución de Ubuntu igual he ejecutado este comando y casi nueve años después tenemos que el número de paquetes ha crecido hasta casi el doble, casi llegamos a 85 mil paquetes, posibles paquetes que podemos instalar en nuestro Ubuntu.

Lo cual nos llega a pensar de qué podría estar pasando ahora en el interior de esos paquetes, de ese código Open Source y si tendría muchas vulnerabilidades o no.

Una vez que ya sabemos la cantidad de paquetes que existen en Linux y la cantidad de paquetes que podemos instalar, es hora de pensar y ponerse a buscar las vulnerabilidades.

Insecure functions

Insecure functions

- In C programming, an insecure function is one that can lead to vulnerabilities due to potential memory management issues or unexpected behavior when used incorrectly.

Insecure function	Safer alternative
<code>strcpy</code>	<code>strncpy</code>
<code>sprintf</code>	<code>snprintf</code>
<code>gets</code>	<code>fgets</code>
<code>scanf</code>	<code>fscanf</code>

Partimos de dos premisas:

- La mayoría del código que podemos encontrar en los repositorios de paquetes de Linux se trata de código escrito en lenguaje C.
- Este lenguaje C puede contener algunas funciones que son potencialmente inseguras.

¿Y qué son las funciones potencialmente inseguras?

Son funciones que de usarlas o que de utilizarla sin la suficiente protección o validación podrían inducir a posibles vulnerabilidades en nuestro código.

Vamos ahora a ver algunos ejemplos de estas funciones inseguras y su alternativa de función que es más segura porque en posteriores versiones del lenguaje C sacaron dicha alternativa de manera oficial.

strcpy

Por un lado tenemos strcpy, que lo que hace esta función es copiar una cadena de caracteres sin comprobar si el destino tenía suficiente espacio asignado para la cadena fuente, es decir, esto podría provocar un buffer overflow.

La alternativa segura es `strncpy`, que lo que hacía este comando es solicitarnos un parámetro extra especificando el tamaño máximo del destino, de esta forma evitamos los posibles buffer overflow.

sprintf

Por otro lado tenemos sprintf, que similar al caso anterior, esto puede provocar un buffer overflow si no se controla correctamente.

Para ello tenemos la alternativa `snprintf` que toma un argumento adicional para indicar el tamaño máximo del buffer.

gets

Por otro lado tenemos la función gets, que lo que hace es leer una línea de entrada del usuario y la coloca en un buffer, esto nuevamente podría provocar un buffer overflow.

Y aquí `fgets` es la alternativa segura y que nuevamente nos permite limitar ese tamaño máximo del buffer.

scanf

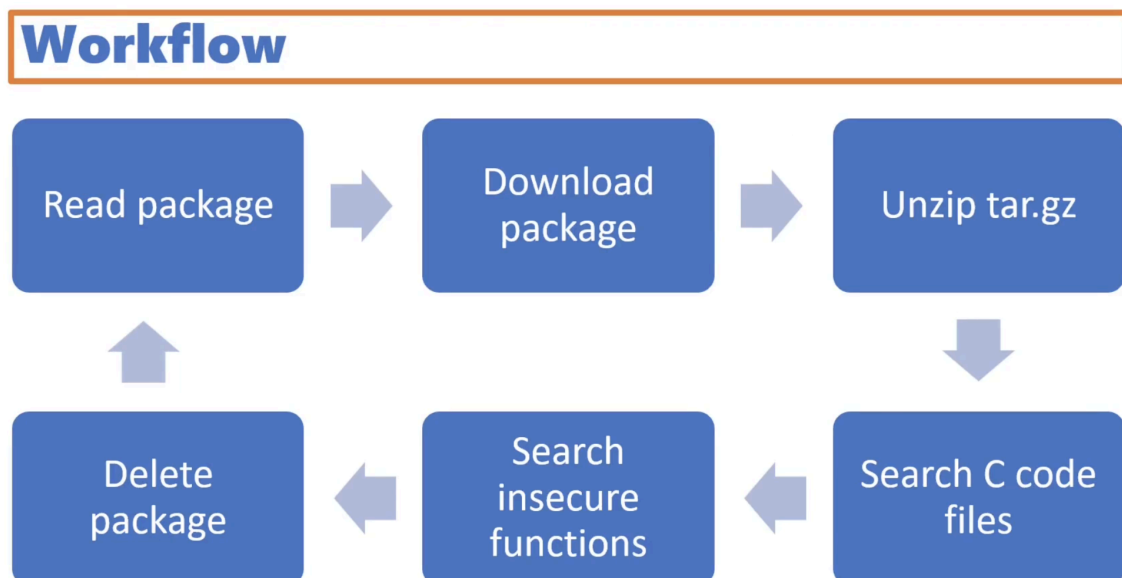
Por último tenemos scanf que permite leer los datos formateados de la entrada y almacenarlos en variables.

Si no validamos dicha entrada lo que podría provocar es un buffer overflow, y esto con **fscanf** lo podemos evitar.

En general, si os dais cuenta lo que hacen estas funciones es aprovecharse de la falta de validación o la falta de sanitización de los datos de entrada para provocar un posible buffer overflow y la alternativa segura lo que hace es limitar esos datos de entrada.

Por tanto podemos decir que estas van a ser las funciones inseguras candidatas que se van a buscar en el código C mediante el uso de OSB-Rastreator.

Workflow



Vamos ahora a hablar del flujo de trabajo que realiza esta herramienta y como podemos ver en la imagen se trata de un bucle porque todo el rato se repite el mismo flujo.

También mencionar que si recordáis al principio vimos esta imagen de abajo y veíamos que aquí se listan todos los paquetes de Linux y que se guardaban en un fichero llamado openHack.txt.

```
@mail:~/perro$ cat get_list_openSource.txt
apt list | cut -d '/' -f1 >> /home/pablo/perro/open2.txt
@mail:~/perro$ wc -l openHack.txt
45494 openHack.txt
@mail:~/perro$
```

La idea ahora, como podéis ver en la primera casilla arriba a la izquierda de la imagen de workflow, es **leer ese paquete** e ir leyendo línea por línea, es decir, paquete a paquete y descargar el código fuente de cada paquete que tenemos guardado en el fichero.

Tras esto el **código se descarga** comprimido en tar.gz y tras esto lo que haremos será **descomprimir el paquete** y buscar entre los ficheros descomprimidos.

Buscaremos ficheros con extensión .c, es decir, ficheros con lenguaje C.

Luego con unas expresiones regulares lo que haremos será **buscar las funciones inseguras** mencionadas anteriormente y almacenaremos los resultados de forma conveniente para su posterior análisis y evaluación.

Por supuesto, si descargamos 45500 paquetes y no los vamos borrando según vamos descargando, nuestro disco se nos quedará pequeño, por eso es importante que vayamos **eliminando ese fichero comprimido** que descargamos y los respectivos ficheros descomprimidos.

```

while read linea
do
    echo $linea
    apt-get source $linea

    #descomprimir
    tars=$(ls *.tar.gz)
    for i in $tars
    do
        tar -xvzf $i
    done

    #descomprimido a buscar
    files=$(find . -name *.c)
    for i in $files
    do
        res=$(cat $i | grep -n [^a-zA-Z,-,_,/]strcpy '(')
        if [ $? -eq 0 ]
        then
            echo >> $HOME/$strcpy/$linea.txt
            echo "$i" >> $HOME/$strcpy/$linea.txt
            echo " $res" >> $HOME/$strcpy/$linea.txt
        fi

        res=$(cat $i | grep -n [^a-zA-Z,-,_,/]gets '(')
        if [ $? -eq 0 ]
        then

```

Aquí podemos ver este workflow que comentamos un poco más en código, en este caso es código C y como podéis ver arriba lo que se hace es un bucle que va leyendo línea por línea, y cada línea es el nombre del paquete a descargar.

Lo que hacemos es ejecutar el comando apt-get source y el paquete el nombre del paquete a descargar.

Esto nos descargará el código fuente en tar.gz, lo descomprimos y posteriormente buscaremos en los ficheros cuya extensión sea C las funciones inseguras como strcpy, gets, etc.

Una vez hecho esto, si se descubre que hay una función potencialmente insegura, como podemos ver en la imagen, lo que se está haciendo es guardar dicha evidencia en algunos ficheros txt para su posterior análisis.

Results

Insecure function	Occurrences
strcpy	9567
sprintf	607
gets	1257
scanf	9426

Imaginaos que ya hemos ejecutado esta herramienta, que ya tenemos los resultados y esta es la tabla de resultados y yo creo que hablan por sí solos.

La verdad que hay bastantes ocurrencias por cada tipo de función potencialmente insegura.

Por un lado tenemos la función strcpy con 9567 ocurrencias en código, es decir, esta función se está utilizando 9567 veces en el código de todos los repositorios y paquetes de Linux, lo cual es bastante.

Por otro lado tenemos sprintf con 607, aquí este número se reduce drásticamente.

También tenemos la función gets 1257 y de nuevo la función scanf con alrededor de 9426 ocurrencias, lo cual también es bastante.

Comentar también que en este análisis se analizaron paquetes con comentarios multilínea y se encontraron 15.647 paquetes con este tipo de comentarios.

¿Y por qué se analiza esto?

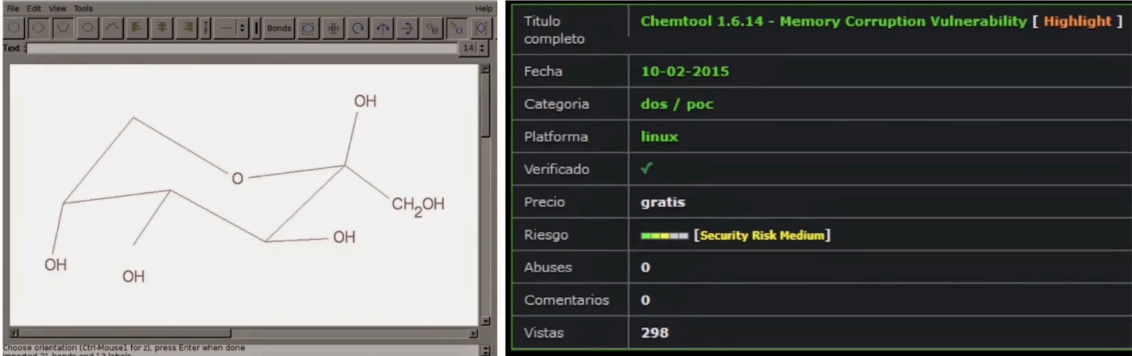
Porque mucho muchas veces el developer deja algunas trazas o alguna información que puede ser útil luego a la hora de buscar vulnerabilidades, y es por ello que se analizó en esta primera prueba de concepto.

Comentar también que por supuesto estos datos pueden contener bastantes falsos positivos, por lo que haría falta una posterior tarea manual donde se comprobaría si

realmente se trata de una posible vulnerabilidad o por el contrario se está validando la entrada de alguna otra forma y no existe riesgo a un buffer overflow.

Para acabar, mencionar que esta herramienta llevó al descubrimiento de una vulnerabilidad cuyo exploit se encuentra en exploit-db y del cual de dicho exploit se hacen echos algunos otros sitios web por Internet.

Results



The screenshot displays the Chemtool 1.6.14 interface. On the left, a chemical structure is shown, which is a complex molecule with multiple hydroxyl groups and a central oxygen atom. On the right, a table lists search results for a vulnerability in Chemtool 1.6.14.

Search Query: text_type: titles vuln_title: Chemtool referencetypes: EXPLOITDB		
ID	Disc Date	EXPLOITDB Title
118250	2015-02-08	36024 Chemtool Input File Handling Memory Corruption DoS Weakness

Chemtool contains a flaw that is triggered as user-supplied input is not properly validated when handling a specially crafted input file. This may allow a context-dependent attacker to corrupt memory and crash the program.

La vulnerabilidad en sí misma se trataba de una corrupción de memoria gracias a la entrada proporcionada en la aplicación Chemtool.

Como podéis ver abajo, se le dio el identificador de 118250 el 8 de febrero de 2015 y ahí se ve claramente que pone Chemtool Input File Handling Memory Corruption DoS Weakness, es decir, es una debilidad que puede negar el servicio gracias a la corrupción de memoria.